Map-Reduce

- "Simplified Data Processing on Large Clusters", by Dean and Ghemawat

Timeline:

- 2003: GFS,
- 2004: MapReduce,
- 2006: Hadoop,
- 2007: HDFS,
- 2014: Spark

Motivation

- Large-scale data processing (petabytes of data)
  - E.g., build search index, or sort, or analyze structure of web
- Seamless scalability
  - Scale "out", not "up"
  - Large cluster of commodity PCs vs small cluster of high-end servers
  - Scale to thousands of CPU cores on clusters of commodity servers
  - The power of scaling out:
    - 1 HDD: 80 MB/sec
    - 1000 HDDs: 80 GB/sec
    - The Web: 20 billion web pages x 20KB ≈ 400 TB
      - 1 HDD: 2 months to read the web
      - 1000 HDDs: 1.5 hours to read the web
- Data locality
  - Move computation to the data
- Applications not written by distributed systems experts
  - Distributing the computations, fault tolerance, recovery, etc.
- Fault tolerance
  - One server may stay up 3 years (1,000 days)
  - For 1,000 servers: 1 fail per day
  - Google had 1M machines in 2011: 1,000 servers fail per day
- Sharing a global state is difficult: synchronization, deadlocks
  - Need a shared nothing architecture
    - Independent nodes
    - No common state
    - Communicate through network and DFS only

MapReduce

- System for automatic parallelizing the computation (batch) of large volume of data across multiple machines
- Provides an API for expressing computations using two operations: Map and Reduce
  - The Map task takes an input file and outputs a set of intermediate (key, value) pairs
  - The intermediate values with the same key are then grouped together and processed in the Reduce task for each distinct key
- Balancing load over servers
  - Full scans of datasets stored in GFS
    - Huge files (100s of GB to TB)
- Data is rarely updated in place

    - Reads and appends are common
- Scalability
    - n "worker" computers get you nx throughput
        - Maps()s can run in parallel, since they don't interact
        - Same for Reduce()s
- The programming model is inspired by functional programming languages
    - Makes easy to distribute computations across nodes
- Transparency
    - Sending application code to servers
    - Tracking which tasks are done
    - Moving data from Mappers to Reducers
- Provides automatic fault tolerance

Examples:

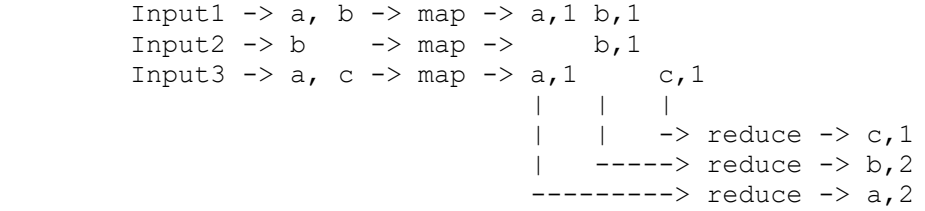
- Word Count: Output frequency of each word

```
Pseudocode:
   var C = map[String, Int]
   for w in words do
      C[w] += 1
   print(C)

MapReduce
  Map(k, v)
    split v into words
    for each word w
      emit(w, 1)

  Reduce(k, v)
    emit(k, len(v))
```

Abstract view



- Input is (already) split into M files
- MR calls map() for each input file, produces set of k2,v2
    - "intermediate" data
    - Each map() call is a "task"
- MR gathers all intermediate v2's for a given k2, and
    - Passes each key + values to a reduce call
- Final output is set of <k2, v3> pairs from reduce()s

- Average-by-key: Given two columns, group rows that share the same value in the first column, then compute the average of the second-column values within each group

```
X,  Y
1, 2.3
2, 3.4
1, 4.5
3, 6.6
2, 3.0
```

```
Pseudocode
      select s.X, avg(s.Y)
      from file as s
      group by s.X


MapReduce:
      Map(loc, line)
            parse the line into a long key and a double value
            emit(key, value)

      Reduce(key, values)
          sum = 0.0
          count = 0
          for each v in values
              count++
              sum += v
          emit(key, sum/count)
```

Execution overview

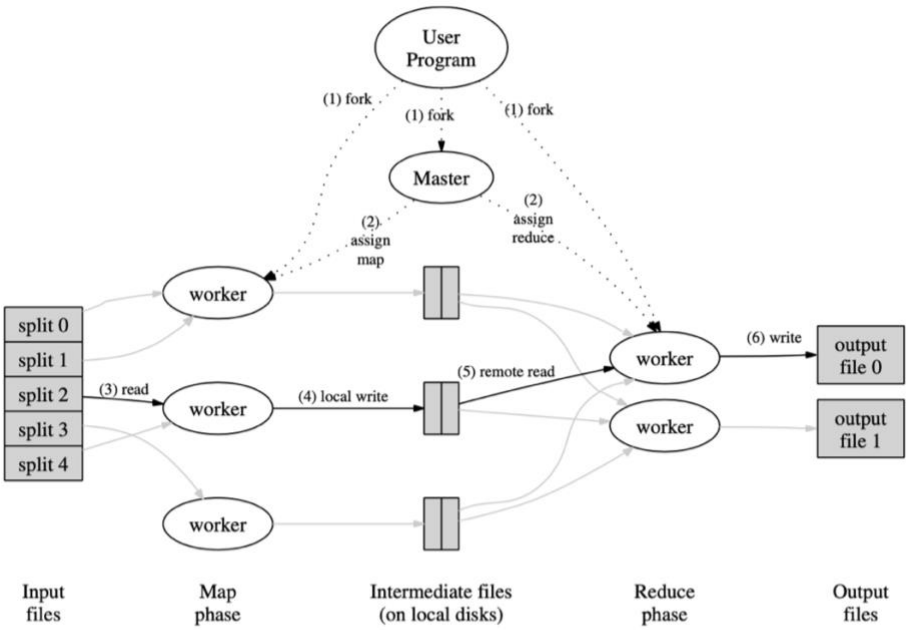


Figure 1: Execution overview

- The user program:
  - Forks a coordinator process and some number of worker processes at different compute nodes
  - Usually, a worker handles either map tasks or reduce tasks, but not both
- The coordinator:
  - Create some number of map tasks and some number of reduce tasks
  - These tasks will be assigned to worker processes by the coordinator
  - Usually, one map task for every chunk of the input file(s), but fewer reduce tasks
  - Keeps track of the status of each map and reduce task (idle, executing at a particular worker, or completed)
- A worker process reports to the coordinator when it finishes a task, and a new task is scheduled by the coordinator for that worker process
  - Each map task:
    - Is assigned one or more chunks of the input file(s) and executes on it the code written by the user
    - Creates a file for each reduce task on the local disk of the worker that executes the map task
  - The coordinator is informed of the location and sizes of each of these files
  - The reduce task:
    - Is assigned by the coordinator to a worker process
    - Given all the files for each unique key
    - Executes code written by the user and writes its output to a distributed file system

Input and output:

- Stored on the GFS cluster file system
- MR needs huge parallel input and output throughput
- GFS splits files over many servers, in 64 MB chunks
  - Maps read in parallel
  - Reduces write in parallel
- GFS also replicates each file on 2 or 3 servers

Network usage:

- Coordinator tries to run each map task on GFS server that stores its input
  - All computers run both GFS and MR workers
  - So input is read from local disk (via GFS), not over network
- Intermediate data goes over network just once
  - Map worker writes to local disk
  - Reduce workers read directly from Map workers, not via GFS
- Intermediate data partitioned into files holding keys

Load balance

- Wasteful and slow if n-1 servers must wait for 1 slow server (straggler) to finish
- But some tasks likely take longer than others

Solution:

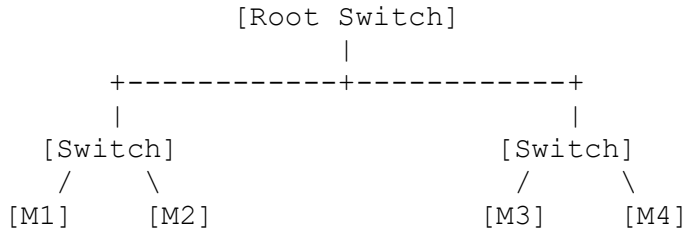
- Many more tasks than workers
  - Coordinator hands out new tasks to workers who finish previous tasks
  - So, no task is too big to dominate completion time (hopefully)
  - So faster servers do more tasks than slower ones, finish about the same time
- Backup tasks
  - When a MapReduce operation is close to completion, the master schedules backup executions of the remaining in-progress tasks
  - The task is marked as completed whenever either the primary or the backup execution completes

Fault Tolerance and Crash Recovery

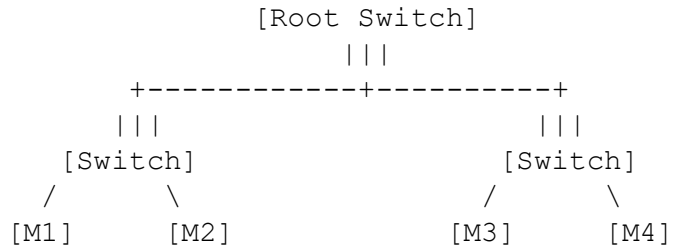
- What if the coordinator crashes?
  - As coordinator failures were rare, their implementation simply aborted the whole execution if coordinator failed, for the execution to be retried from the start
- Map worker crashes
  - Coordinator notices worker no longer responds to pings
  - Coordinator knows which Map tasks it ran on that worker
    - Those tasks' intermediate output is now lost, must be recreated
    - Coordinator tells other workers to run those tasks
  - Can omit re-running if reducer already fetched the intermediate data
  - What if the coordinator gives two workers the same Map() task?
    - Perhaps the coordinator incorrectly thinks one worker died
    - It will tell Reduce workers about only one of them
- Reduce worker crashes
  - Finished tasks are OK
    - Stored in GFS, with replicas
  - Coordinator re-starts worker's unfinished tasks on other workers
  - What if the coordinator gives two workers the same Reduce() task?
    - They will both try to write the same output file on GFS!
    - *Atomic GFS rename* prevents mixing
      - One complete file will be visible
- What if a worker computes incorrect output, due to broken h/w or s/w?
  - Too bad! MR assumes "fail-stop"(after a node crashes, it never recovers) CPUs and software

Issues

- No interaction or state (other than via intermediate output)
- No multi-stage pipelines
- No real-time or streaming processing
- In 2004, authors were limited by network capacity
  - Maps read input from GFS
  - Reducer read map output
    - Can be as large as input, e.g., for sorting
    - Reduces write output files to GFS



  - In MR's all-to-all shuffle, *half the of traffic* goes through root switch
  - Paper's root switch: 100 to 200 gigabits/second, total
    - 1800 machines, so 55 megabits/second/machine
    - 55 is small, e.g., much less than disk or RAM speed
- Today: networks and root switches are much faster relative to CPU/disk



Current status

- Hugely influential (Hadoop, Spark, etc.)
- Probably no longer in use at Google
  - Replaced by Flume/FlumeJava (see paper by Chambers et al)
  - GFS replaced by Colossus, and BigTable

Conclusion

- MapReduce single-handedly made big cluster computation popular
- Scales well
- Easy to program
- Failures and data movement are hidden
- Not the most efficient or flexible
- One of the top paying technologies (2022)

Optional Read:

- MapReduce: simplified data processing on large clusters
  - https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/
- MapReduce: A major step backwards, by David J. DeWitt and Michael Stonebraker (2007)
  - https://homes.cs.washington.edu/~billhowe/mapreduce_a_major_step_backwards.html
- Databases are hammers; MapReduce is a screwdriver, by Mark C. Chu-Carroll
  - https://scienceblogs.com/goodmath/2008/01/22/databases-are-hammers-mapreduc